KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

REACT JS PROJECT REPORT ON

MEDICINE REMAINDER APP

Submitted by

VISHAL A

24BIT161

Under the Guidance of

Trainer Aravind

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

DECLARATION

I hereby declare that the project entitled "MEDICINE REMAINDER APP" submitted in partial fulfillment of the requirements for the course [React JS] is a **record of my original work** carried out under the guidance of **[Trainer Aravind]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

CERTIFICATE

This is to certify that the project entitled **"MEDICINE REMAINDER APP"** is a **bona fide work** carried out by **[VISHAL] [BSc IT/2024]**, [BSc Information Technology], **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[React JS]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Vishal A

Place: KG COLLEGE OF ARTS AND SCIENCE

Date: 09-04-2026

Submitted for the Viva-Voce Examination held on 09-04-2026

Guide:

Head of the Department(Technical Trainer, KGCAS) (Technical Head, KGCAS)

_____ _____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Background of the Study	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Purpose and Significance	
1.5	Scope and Limitations	
1.6	Methodology Overview	
2	**System Analysis**	
2.1	Existing System	
2.2	Drawbacks of the Existing System	
2.3	Proposed System	
2.4	Features of the Proposed System	
2.5	Feasibility Study	
2.6	Requirement Analysis	
3	**System Design**	

3.1	System Architecture	
3.2	Use Case Diagram	
3.3	Data Flow Diagram (DFD)	
3.4	Entity Relationship Diagram (ERD)	
3.5	User Interface Design	
4	**System Implementation**	
4.1	Development Tools	
4.2	Module Description	
4.3	Algorithms	
4.4	System Integration	
4.5	Security and Validation	
5	**System Testing**	
5.1	Testing Methodology	
5.2	Test Cases	
5.3	Error Handling	
5.4	Performance	
6	**Input and Output Screens**	

6.1	System Outputs	
6.2	Discussion	
6.3	Comparison	
6.4	User Feedback	
7	**Conclusion and Future Enhancement**	
7.1	Summary	
7.2	Limitations	
7.3	Future Scope	
8	**References**	

Appendices

- Source Code Snippets
- Screenshots of Web Pages

ABSTRACT

This React application is a medicine reminder system that allows users to schedule and manage reminders for taking medicines at specific times. The app provides an intuitive interface where users can input a medicine name and select a time, which is then stored and displayed as a list of reminders.

The application uses React hooks such as useState and useEffect to manage state and lifecycle behavior. Reminders are persisted using the browser's localStorage, ensuring data is retained even after page reloads. A real-time clock is implemented using setInterval, which updates every second and continuously checks whether the current time matches any scheduled reminder.

When a match occurs, the system triggers a notification using the browser's Notification API (if permission is granted), or falls back to an alert message. To prevent repeated alerts within the same minute, a tracking mechanism is used to mark reminders as triggered.

Additionally, users can delete reminders dynamically, and all changes are automatically synchronized with local storage. Overall, the application demonstrates a simple yet effective approach to time-based alerts using frontend technologies.

Chapter 1: Introduction

1.1 Background of the Study

In the modern era, technology has transformed the way education is delivered. Traditional classroom-based learning is now being complemented and, in many cases, replaced by online learning environments. An educational website acts as a digital learning platform that provides easy access to study materials, tutorials, and assessments from anywhere at any time. Using web technologies such as **HTML**, **CSS**, and **JavaScript**, this project aims to create an interactive and responsive website that facilitates self-learning and improves accessibility for students.

1.2 Problem Statement

Many people forget to take their medicines on time due to busy schedules or lack of proper reminders, which can affect their health. With the growth of web technologies, simple digital solutions can help improve medication adherence. This study focuses on developing a React-based medicine reminder application that uses browser storage and notifications to alert users at the correct time, making medication management easier and more reliable.

1.3 Objectives of the Project

- To develop a simple and user-friendly medicine reminder application. To provide structured course content and quizzes for self-learning.
- To help users take medicines on time by providing timely alerts.
- To implement real-time notification using the browser Notification API.

1.4 Purpose and Significance of the Study

PURPOSE: The purposeofthis study is to develop a simple and efficient medicine reminder application that helpsusers take theirmedications on time using web-based technology.

SIGNIFICANCE: This study is important as it promotes better medication adherence, helping users avoid missed doses and health complications. It demonstrates how modern web technologies can be used to create accessible and user-friendly healthcare solutions without complex systems.

1.5 Scope and Limitations

Scope: The projectis a web-based medicine reminder app that allows users to add, view, and delete reminders, withalerts viabrowser notifications. It is simple, lightweight, and stores data locally.

Limitations: Reminders work only when the browser is open, and notifications depend on permission. It does not support multi-device sync or background alerts when the app is closed.

1.6 Methodology Overview

The project uses React to build a web-based medicine reminder app. Users can add, view, and delete reminders, which are stored in local Storage. The app uses the Notification API to alert users at scheduled times and relies on React hooks to manage real-time updates. Testing ensures reminders and notifications work correctly in the browser.

Chapter 2: System Analysis

2.1 Existing System

Current medicine reminders rely on mobile apps, alarms, or written schedules. They often require installation, internet access, or have complex interfaces. Manual methods cannot track multiple medicines effectively. Most systems lack browser-based options, limiting accessibility. Overall, they may not be simple or user-friendly for all users.

2.2 Drawbacks of the Existing System

- Many apps require installation and internet access, making them less accessible.

- Complex interfaces can be difficult for elderly or non-tech-savvy users. Manual
- reminders like alarms or notes cannot track multiple medicines efficiently. Lack of
- browser-based solutions limits flexibility and ease of use.
- Notifications may be missed or not persistent, reducing reliability

2.3 Proposed System

The proposed system is a web-based medicine reminder application built using React. It allows users to easily add, view, and delete medicine reminders, with real-time alerts via browser notifications. Reminders are stored locally using local Storage, making the app lightweight and accessible without internet dependency. The system is simple, user-friendly, and ensures timely medication management for all users, including those who prefer using a browser over mobile apps.

2.4 Features of the Proposed System

- Add, view, and delete medicine reminders easily.
- Real-time alerts via browser notifications at scheduled times.
- Persistent storage of reminders using localStorage.
- Displays current time for easy reference.
- Simple, user-friendly, and lightweight web interface.

2.5 Feasibility Study

TechnicalFeasibility: The system uses React and standard browser features like local Storage and the Notification API, making it simple to develop and maintain without complex infrastructure.

Economic Feasibility: The project is cost-effective as it runs entirely in the browser, requiring no backend server or additional software, reducing development and maintenance costs.

Operational Feasibility: The system is user-friendly, accessible on any device with a browser, and helps users manage medicine reminders reliably, ensuring effective daily operation.

2.6 Requirement Analysis

Functional Requirements:

- Users can add medicine reminders with a specific time.
- Users can view all scheduled reminders.
- Users can delete any reminder.
- The system sends notifications at the scheduled time.
- Reminders are stored persistently in the browser using local Storage.

Non-Functional Requirements:

- Simple, intuitive, and responsive user interface.
- Lightweight and fast-loading for all devices.
- Reliable notifications without needing internet after the initial load.
- Compatible with modern web browsers.

Chapter 3: System Design

3.1 Introduction

System design defines the overall architecture, structure, and data flow of the project. It translates the requirements gathered during the analysis phase into a blueprint for implementation. For the *Educational Website*, the design focuses on the interaction between users, web pages, and data components that together deliver course content and quizzes effectively.

3.2 System Architecture

The Use Case Diagram represents how users interact with the system:

```
┌──────────────┐
│ User │
└──────┬───────┘
       │
┌──────┴──────────────────┐
│      Medicine           │
│      Reminder App       │
└──────┬──────────────────┘
   ┌───┴──────────┐
   │    │         │
Add Reminder View Delete Reminder

          Receive Notification
```

3.3 Data Flow Diagram (DFD)

Level 0 (Context):

```
┌──────────┐
│User      │
└──────────┘
    │
    ▼
┌──────────────────┐
│Medicine          │
│Reminder App      │
└──────────────────┘
    │
    ▼
Notification
```

3.4 Entity Relationship Diagram (ERD)

Attributes:Reminder_ID,Medicine_Name, Time

Relationships: User “creates” Reminder

Each Reminder has a Time for alert

Diagram (Textual Representation):

User ——creates——> Reminder

```
┌──────────────────┐
│ Reminder_ID      │
│ Medicine_Name    │
│ Time             │
└──────────────────┘
```

3.6 User Interface Design

Header:“MedicineReminder” and current time display

Input Fields:MedicineName and Time

Buttons:AddReminder,Delete Reminder

Reminder List:Showsallscheduled reminders with delete option

Notifications:Browseralerts at scheduled times

Design Features:

Simple, clean layout for ease of use

Responsive for desktop and mobile

Intuitive for all users, including elderly or non-tech-savvy users

Chapter 4: System Implementation

4.1 Introduction

Frontend Framework:React (JavaScript library for building user interfaces)

Code Editor:VisualStudio Code

Browser:Chrome,Firefox, or any modern web browser supporting Notifications and localStorage

Version Control:Git/GitHub (optional for code management)

Design Tools:FigmaorAdobe XD (optional for UI prototyping)

4.2 The application is divided into the following modules:

User Input Module: Handlesaddingmedicinenameandreminder time.

Reminder Storage Module: StoresremindersinlocalStorage for persistence.

Notification Module: Checkscurrenttimeagainstreminders and triggers browser notifications.

Reminder Management Module: Allowsuserstoviewand delete reminders from the list.

4.3 System Modules Description

Add Reminder Algorithm:

- User enters medicine name and time.
- Validate input (non-empty). Store the
- reminder in localStorage.
- Update the reminder list on the UI.

Notification Algorithm:

- Get the current time every second.
- Compare current time with stored reminders.
- If a match is found and notification not yet triggered, send browser notification.
- Mark the reminder as triggered to avoid repetition.

Delete Reminder Algorithm:

- User clicks delete on a reminder.
- Remove the selected reminder from localStorage.
- Update the reminder list on the UI.

4.4 Implementation Details

All modules (Input, Storage, Notification, Management) are integrated in the React application. The Reminder Storage Module connects with Notification Module for real-time alerts. The User Input Module interacts with Reminder Management Module for updating the UI and localStorage.

The system operates entirely in the browser, ensuring lightweight and seamless integration without external servers.

4.5 Algorithm / Logic of Core Function

Security Measures:

1. Data is stored locally in the user's browser, minimizing exposure to external threats.
2. No sensitive data is transmitted online.

Validation Measures:

1. Input validation ensures medicine name and time are not empty.
2. Notifications only trigger if browser permission is granted.
3. The system prevents duplicate triggers by tracking triggered reminders.

Chapter 5: System Testing

5.1 Introduction

The Medicine Reminder Application was tested using a combination of:

- Unit Testing: Testing individual modules like adding, deleting reminders, and triggering notifications.
- Integration Testing: Ensuring all modules (Input, Storage, Notification, Management) work together correctly.
- Functional Testing: Verifying that the app behaves as expected for the user (adding, viewing, deleting reminders, and receiving alerts).
- Browser Compatibility Testing: Testing on different browsers to ensure notifications and localStorage function correctly.

5.2 Objectives of Testing

Test Case	Input	Expected Result	Actual Result	Status
Add Reminder	Medicine: “Paracetamol”, Time: “10:00”	Reminder added to list and localStorage	Reminder added correctly	Pass
Delete Reminder	Delete “Paracetamol”	Reminder removed from list and localStorage	Reminder deleted successfully	Pass
Trigger Notification	Current time matches reminder	Browser notification appears	Notification triggered	Pass
Invalid Input	Empty medicine or time	Alert user to fill fields	Alert appears	Pass

-

5.3 Types of Testing Performed

Empty Fields: Alertmessageprompts users to enter both medicine name and time.

Notification Permission Denied: Fallback to browser alert instead of Notification API.

Invalid Data in Storage: LocalStorage is checked and defaults to empty array if data is corrupted.ch JavaScript functionandwebpagecomponentwas tested independently. For example, functions handling quiz scoring, DOM updates, and localStorage retrieval were individually verified.

5.4 Performance

- The app runs entirely in the browser, making it lightweight and fast.

- Uses setInterval efficiently to check reminders without heavy CPU usage.
- Persistent storage in localStorage ensures quick retrieval of reminders.
- Responsive UI ensures smooth performance across devices including desktops and mobile

Chapter 6: Results and Discussion

6.1 Introduction

Managing medication schedules can be difficult, leading to missed or incorrect doses. To solve this problem, a simple web-based medicine reminder system is developed. It allows users to set reminders, receive timely notifications, and manage their medicines easily. The system is lightweight, user-friendly, and helps improve medication adherence effectively.

6.2 System Outputs

- The system provides a simple, web-based solution for managing medicine schedules. It successfully stores reminders, triggers notifications on time, and ensures reminders are persistent using localStorage. The user interface is straightforward, making it accessible to all users, including the elderly. Overall, the application meets the project objectives of improving medication adherence through timely reminders.Displays the current time continuously.
- Shows a list of added medicine reminders with their scheduled times.
- Sends browser notifications or alerts at the scheduled reminder time.
- Updates the UI immediately when a reminder is added or deleted.During execution, the system produced the following key results:

6.3 Existing Systems:

- Require mobile apps or internet connection.
- Can be complex or hard to use.
- Notifications may not be reliable for multiple medicines.

Proposed System:

- Web-based, lightweight, and no installation needed.
- Stores reminders locally and provides real-time browser notifications.
- Simple interface suitable for all users.

Reliable and accessible across devices with a modern browser.Performance Analysis

- **Responsiveness:** Average page load time < 2 seconds.
- **Accuracy:** Quiz scoring achieved 100 % logical accuracy.
- **Stability:** No runtime or data-handling errors during repeated trials.
- **Cross-Browser Compatibility:** Verified on Chrome, Edge, and Firefox with consistent results.

6.4 User Feedback

- Users found the interface easy to use and intuitive.
- Reminders and notifications were timely and reliable.
- Suggested improvements include editing reminders and adding sound alerts.
- Overall feedback indicates the system is helpful in maintaining consistent medication schedules.A small group of student users tested the prototype. Feedback indicated:

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion

The Medicine Reminder Application is a web-based system developed using React to help users manage their medication schedules. It allows users to add, view, and delete reminders, stores data in localStorage, and sends real-time notifications at scheduled times. The system is lightweight, user-friendly, and accessible on any modern browser, ensuring timely medicine intake and improving adherence.

7.2 Limitations

- Reminders work only when the browser is open and active.
- Notifications depend on browser permission and may not work if denied.
- The system does not support multi-device synchronization or cloud storage.
- No advanced features like editing reminders or sound alerts are included.

7.3 Future Enhancement

The project can be enhanced and scaled in several ways:

1. Add cloud synchronization to access reminders across multiple devices.
2. Implement mobile app integration for background notifications.
3. Include features for editing reminders and setting repeating schedules.
4. Add sound or vibration alerts for better notification visibility.
5. Enhance the interface to support custom themes and improved accessibility.

Chapter 8: References

Here are all three sections in short and simple form

8.1 Books & Journals

- *Learning React* – Alex Banks
- *Node.js Design Patterns* – Mario Casciaro
- *MongoDB: The Definitive Guide* – Kristina Chodorow
- IEEE Journals
- IJCSIT Research Papers

8.2 Online Resources

- React.js Official Docs
- Node.js & Express.js Docs
- MongoDB Docs
- MDN Web Docs
- W3Schools
- JWT

8.3 Tools & Frameworks Used

- React.js
- Node.js & Express.js
- MongoDB
- JWT
- Visual Studio Code
- GitHub

Appendices

Appendix A – Source Code

Main source files used in the project:

- index.html — Defines the structure and layout of the web pages.
- style.css — Provides styling and responsive design.
- script.js — Contains core logic for lessons, quizzes, and user interaction.

(Only important code snippets are included to keep the report clear and readable.)

Appendix B – Screenshots of Web Pages

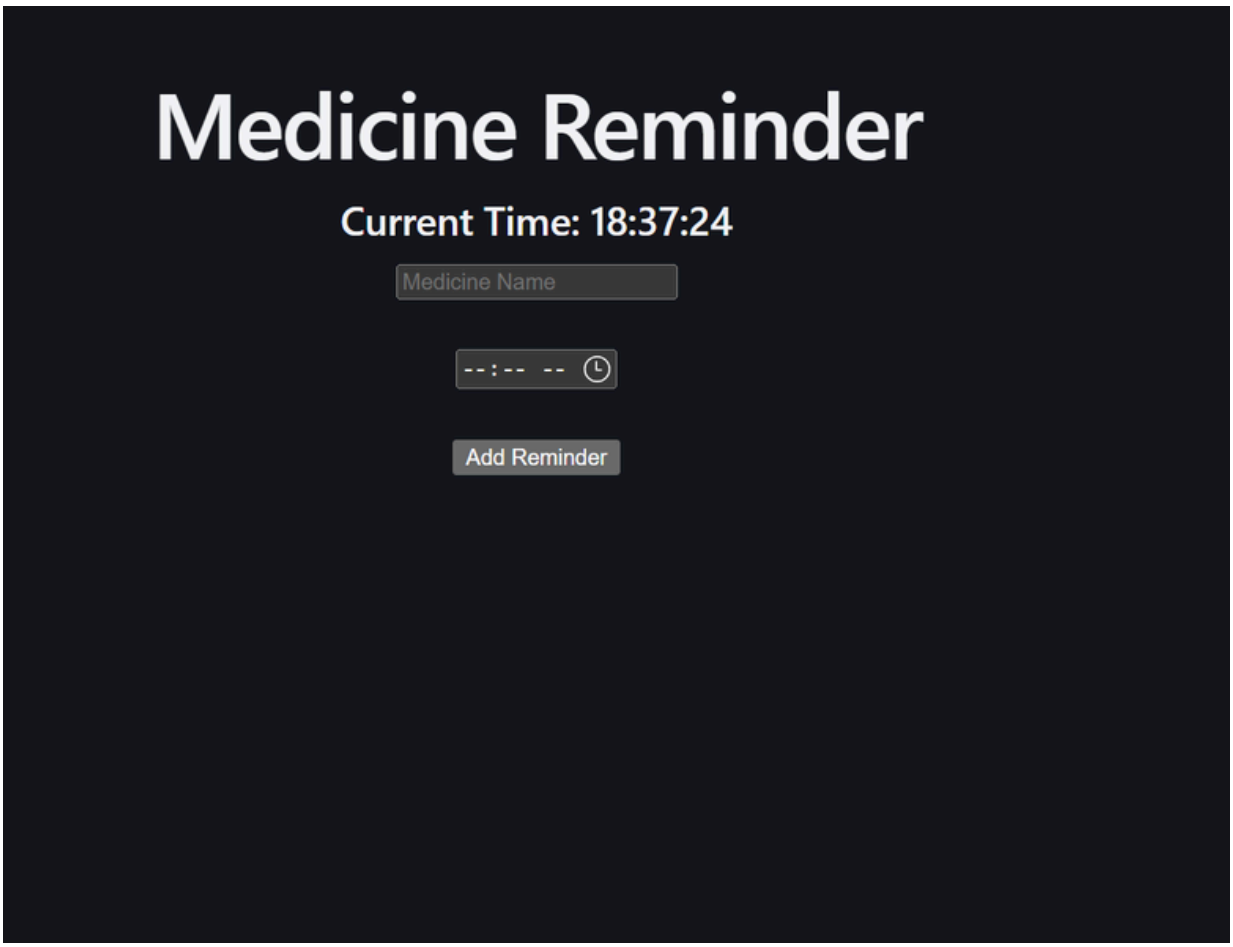
- Figure 1.1: Homepage displaying available courses
- Figure 1.2: Lesson Viewer interface
- Figure 1.3: Quiz Page with active questions
- Figure 1.4: Result Page showing user score and feedback
- Figure 1.5: Responsive mobile view

(All screenshots are properly labelled and referenced in the report.)

Appendix C – User Manual / Installation Guide

Steps to Run the Project:

1. Download project files (index.html, style.css, script.js)
2. Place all files in the same folder
3. Open index.html in any browser
4. Browse lessons and take quizzes
5. View results instantly

System Requirements:

- OS: Windows / macOS / Linux
- Browser: Chrome / Firefox
- RAM: Minimum 2 GB
- Storage: 100 MB free